

FinOps Cloud Data Platform

Provisional Thesis Extract

29 September 2026

Current manuscript: ten chapters and bibliography.
Planned figures, unfinished appendices and working registers are excluded.
This extract is provided for reading and layout review.

Contents

1. General Introduction and Professional Context	4
1.1 - Professional Context and Stakeholder Roles	4
1.2 - Current Process and Diagnosis	4
1.3 - Problem Statement and Research Questions	5
1.4 - Objectives, Scope and Constraints	6
1.5 - Summary of Requirements	6
1.6 - Thesis Structure	7
2. FinOps and Cloud Economics	8
2.1 - FinOps as a Business and Operational Framework	8
2.2 - Inform, Optimize and Operate in the Project	8
2.3 - Cloud Pricing and Cost Mechanisms	9
2.4 - Allocation, Showback and Governed Self-Service	11
3. Methodology and Evaluation Framework	13
3.1 - Objectives and Methodological Framework	13
3.2 - Experimental Dataset Preparation	14
3.3 - Data Quality, Governance and Change Control	15
3.4 - Progressive Implementation and Automation	16
3.5 - Analytical Preparation	16
3.6 - FinOps Data Product Delivery and Operation	17
4. General Design and Architecture Choices	18
4.1 - Design Requirements and Principles	18
4.2 - General Platform Design (Logical HLD)	19
4.3 - Technology Evaluation and Selection	21
4.4 - Selected Physical Architecture	23
4.5 - Data Architecture and Dimensional Modeling	26
5. Detailed Design and Implementation	29
5.1 - Independent FOCUS Data Generator	29
5.2 - Local End-to-End Proof of Concept	30
5.3 - Databricks Cloud Data Platform	31
6. FinOps Analysis of Results	35
6.1 - Scope and Method for FinOps KPI Analysis	35
6.2 - Analysis of Costs and Pricing Mechanisms	36
6.3 - Allocation and Resource Analysis	37
7. Technical Validation and Performance	38
7.1 - Local Datamart and Dashboard Validation	38
7.2 - Existing Power BI Diagnosis and Measurement Protocol	39
7.3 - Architecture Comparison and Performance Levers	40
7.4 - Databricks/GCP Validation	40

8. Economic Analysis of the Solution	43
8.1 - Costing Scope and Method	43
8.2 - Build-Cost Scenarios	43
8.3 - Operating-Cost Scenarios	44
8.4 - Applying FinOps to the FinOps Platform	46
9. Discussion, Limitations and Recommendations	49
9.1 - Discussion, Limitations and Recommendations	49
9.2 - Potential Contribution of a Data Engineer Embedded in FinOps	50
10. General Conclusion and Outlook	52
10.1 - General Conclusion	52
10.2 - Learning Outcomes and Application of Master's Coursework	53
10.3 - Future Work	53
Bibliography	54

1. General Introduction and Professional Context

1.1 - Professional Context and Stakeholder Roles

This project examines the production and use of cloud-cost information within the host organization's FinOps reporting process. The project framing describes a central Data Platform function, a Power BI team and a separate FinOps team. FinOps defines the financial questions and reviews the resulting indicators; the platform and reporting functions provide the technical means to prepare and present them. These responsibilities explain why a change to a cost measure can involve several teams even when the underlying data is already available.

The existing Cloud Showback dashboard serves FinOps, its management and managers in other domains, according to the student's account. The issue examined here is therefore the reliability and adaptability of the information supplied through that process, together with the degree of analytical autonomy available to authorized users. Section 1.2 describes the reported change workflow.

1.2 - Current Process and Diagnosis

The student reports that the current Power BI dashboard is used by FinOps, its management and managers of domains and subdomains in other teams. Dashboard access is therefore already part of the current process. The remaining question is whether each group can obtain the analyses it needs within its authorized scope and how changes to those analyses are made. Effective permissions and unmet requests have not yet been documented.

In the change workflow described by the student, a supervisor reviews the dashboard and provides feedback. The manager relays requested changes to an external contributor, who implements them. The student reports that these handoffs lengthen corrections. No request log, elapsed-time series or iteration count is currently available to quantify that effect. The contributor's location describes the service arrangement; it does not explain the delay by itself.

This account identifies a process dependency that can be tested. A request may require business approval, technical interpretation, implementation and acceptance, each with its own waiting time. The analysis in section 9.2 considers whether closer Data Engineering support within FinOps could reduce some handoffs while preserving review and publication controls. That proposal remains an organizational hypothesis until comparable requests and effort are measured.

1.3 - Problem Statement and Research Questions

The project addresses the transformation of detailed FOCUS cost and usage records into reliable data products. Those products must support several consumers while preserving shared definitions, access control, traceability and confidentiality. The reported dashboard-change workflow makes the way those products are maintained and used part of the problem.

The research question is:

How can a FinOps data platform transform FOCUS cost and usage data into reliable data products and provide governed self-service analytics to managers and application owners while supporting Inform, Optimize and Operate?

1.3.1 Three complementary subquestions

The central question is divided into three complementary subquestions rather than independent projects. They reflect the initial framing and organize the thesis argument.

1. Platform reliability: How can FOCUS cost and usage data become reliable, traceable and reusable data products while addressing security and scalability requirements?
2. Governed self-service: How can managers and application owners explore their authorized scope without manual FinOps extracts while preserving confidentiality and consistent indicators?
3. FinOps decision support: How can these products improve cost visibility, support analysis of optimization opportunities and enable recurring Inform, Optimize and Operate activities?

The first will be assessed through contract and quality controls, lineage, reconciliation and reproducibility, without conflating local execution with target-platform validation. The second requires user scenarios and access tests for every exposed channel; a usable interface alone does not establish secure self-service. The third requires correct and useful KPIs, separating cost differences, recommendations and measured savings.

1.3.2 Scope of self-service

The project aims for centralized FinOps data production and distributed consumption according to user permissions. Independent access does not require each domain to produce and maintain its own data. Section 4.2 develops this position and its relationship to Data Mesh; chapter 9 discusses its implications.

1.3.3 Additional design questions

Supporting questions address:

- the balance between Data Engineering depth, governance, cost and feasibility;

- the Data Contract and controls required to establish trust;
- the design of ownership, RLS and analyst access;
- the cost measures that support realized rate comparisons;
- the additional data required for potential-savings recommendations.

1.4 - Objectives, Scope and Constraints

The project aims to turn FOCUS cost-and-usage files into traceable analytical products. Its technical objectives are to validate a versioned data contract, preserve source lineage, reconcile charges across Bronze, Silver, Gold and datamarts, and make the resulting measures available to a reporting application. The evaluation also examines whether users can answer defined FinOps questions from these products and whether the platform records enough information to diagnose a failed or repeated run.

The empirical dataset begins with anonymized Azure FOCUS files and extends them synthetically. One local DuckDB and Streamlit prototype supports January 2025 validation. The active cloud implementation uses Databricks on GCP with separate DEV and PROD catalogs. These settings serve different tests; a local result cannot establish cloud behavior. The enterprise Azure architecture considered in the initial framing remains a design context rather than an implemented deployment.

Confidentiality prevents publication of the organization's detailed billing history. The limited reference data and synthetic extension constrain business conclusions, especially claims about savings or forecasting. Time, available cloud resources and incomplete access evidence constrain the performance and security evaluation. Section 3.4 defines the evaluation protocol; chapter 7 distinguishes completed tests from those still required.

1.5 - Summary of Requirements

The project framing groups ten acceptance requirements under platform reliability, governed self-service and FinOps decision support. Reliability requires identifiable historical and incremental batches, a versioned FOCUS contract, traceable Bronze-to-Gold transformations, reconciliation and repeatable execution. The data products must record their definitions, owner, freshness and quality before they are presented as certified within the project.

Governed self-service requires more than a working dashboard. The proposed acceptance scenario asks a manager to answer cost questions for an authorized domain and an analyst to produce an additional aggregation from a certified datamart. Access tests must cover the dashboard, exports and direct queries; filtering one interface does not secure another. These scenarios remain acceptance criteria until representative identities and permissions have been tested.

Decision support covers showback, period comparisons, documented cost formulas, pipeline status and recovery. The requirements distinguish a calculated price difference from a verified saving and require unknown allocation to remain visible. Chapter 7 reports what the prototype establishes against these criteria. The complete requirement matrix and its proposed evidence are retained in the project framing document.

1.6 - Thesis Structure

Chapter 2 defines the FinOps and cloud-cost concepts used in the analysis. Chapter 3 derives required artifacts from the intended outcomes, then explains dataset preparation, quality and governance, progressive implementation, analytical preparation and data-product delivery. It defines acceptance criteria without assuming that every proposed test has succeeded. Chapter 4 presents the logical and physical architecture, technology choices and analytical data model. Chapter 5 follows the three implementation workstreams: the independent FOCUS generator, the local DuckDB/Streamlit proof of concept, and the Databricks cloud platform. This ordering explains the final system; it is not a chronology of development. Chapter 6 interprets the available cost and allocation results. Chapter 7 assesses local validation and the evidence currently available for Databricks, including the limits of a performance comparison.

Chapter 8 examines build and operating costs and applies FinOps reasoning to the platform's own workloads, including the later workspace and compute decisions. Chapter 9 discusses the findings, limitations and organizational options. Chapter 10 returns to the research question and identifies work that depends on further evidence. Throughout the thesis, observed results are separated from reported execution and proposed tests.

2. FinOps and Cloud Economics

2.1 - FinOps as a Business and Operational Framework

FinOps provides the business framework for interpreting the project's cost data. A pipeline can reconcile charges technically while leaving allocation, ownership or business meaning unresolved. The platform therefore supplies data for financial questions as well as for reporting.

The FinOps Foundation framework defines FinOps through collaboration between engineering, finance and business functions, timely data and financial accountability. The project applies this framework to a bounded problem: preparing governed cost products and examining what decisions those products can support. It does not evaluate the maturity of the organization's full FinOps practice.

Why cloud economics require FinOps

Cloud expenditure varies with usage, configuration, location and contractual terms. Resource operators, budget owners and invoice reviewers may therefore need different views of the same charge. Shared cost definitions and explicit allocation rules allow these views to be compared without treating every difference as waste or saving.

Roles in the proposed operating model

- The Data Platform function provides ingestion, transformation, quality, security and operational controls.
- FinOps owns cost semantics, allocation rules, financial indicators and business validation.
- The Power BI or dashboard function owns presentation and semantic-model standards.
- Managers and application owners consume information for their authorized perimeter.
- Authorized analysts may need direct access to governed datamarts for additional analysis.

This is a design allocation of responsibility. The existing dashboard's actual permission and change processes are described from the student's account in section 1.2; the proposed roles require organizational validation before they can be reported as established practice.

2.2 - Inform, Optimize and Operate in the Project

Analytical positioning

The FinOps Foundation describes Inform, Optimize and Operate as recurring phases. This thesis uses them to organize the questions tested by the prototype, while recognizing that a reliable cost measure can inform several activities at once.

Inform

Inform establishes visibility and accountability. It includes cost trends, allocation, showback, comparison between periods, identification of unallocated expenditure and access to consistent definitions. In the prototype, Inform is represented principally by the certified datamarts and the Streamlit dashboard.

Optimize

Optimize evaluates whether expenditure delivers sufficient value and whether rates, commitments or resource usage can be improved. The current dataset supports comparisons among list, contracted, effective and billed cost. It does not by itself support defensible rightsizing recommendations because those require utilization, inventory, recommendation or pricing-reference data.

Operate

Operate embeds FinOps into recurring processes. For this PFE it includes ingestion, data quality, reconciliation, freshness, execution traceability and recovery. Forecasting and anomaly detection remain conditional on sufficient historical data and an explicit evaluation protocol.

Boundary of claims

The dashboard demonstrates cost visibility on synthetic data. It does not demonstrate measured organizational savings, a reduction in company workload or production-grade continuous operation. These claims require separate evidence.

2.3 - Cloud Pricing and Cost Mechanisms

From consumption to a financial charge

A cloud charge results from the interaction of a consumed service, a pricing quantity and unit, a public rate, contractual conditions, commitment benefits, credits, corrections, taxes and billing timing. Consequently, the word “cost” is ambiguous unless the selected cost perspective and accounting basis are stated.

At line level, the simplified pricing path used for interpretation is:

```
Consumed resource or service
    v conversion to pricing quantity
Public list price
    v negotiated contractual reductions
Contracted price
    v allocation of commitment benefits and amortization
Effective cost
    v invoice timing and covering-charge treatment
Billed cost
```

This diagram is explanatory. The equality and ordering of these values cannot be assumed for every charge category, credit, correction, tax or purchase row.

Four FOCUS cost perspectives

Measure	Business interpretation	Main use in the project
ListCost	Cost calculated from the provider's public list unit price and pricing quantity	Reference for rate-comparison analysis
ContractedCost	Cost calculated from the negotiated contracted unit price and pricing quantity	Isolate the effect of negotiated pricing where comparable
EffectiveCost	Economically allocated cost after relevant discounts and amortized commitment purchases	Accrual-oriented service and resource analysis
BilledCost	Charge serving as the basis for invoicing in the billing period	Invoice reconciliation, cash-oriented reporting and allocation

These definitions follow the FOCUS 1.0 specifications for List Cost, Contracted Cost, Effective Cost and Billed Cost. BilledCost and EffectiveCost answer different questions. A commitment purchase may be billed at one time while its economic cost is allocated to covered usage over time. Their difference must therefore not automatically be labelled as a saving.

Pricing quantity and unit price

Where the required fields are available and comparable:

$$\begin{aligned}\text{ListCost} &= \text{ListUnitPrice} \times \text{PricingQuantity} \\ \text{ContractedCost} &= \text{ContractedUnitPrice} \times \text{PricingQuantity}\end{aligned}$$

Pricing quantity can differ from consumed quantity because providers may apply block, tier or normalized pricing. Corrections are also capable of breaking a naive equality at individual-row level and must be treated according to their charge class.

Negotiated and commitment-related effects

The following decompositions are analytical candidates, not universally valid accounting identities:

$$\begin{aligned}\text{Negotiated rate effect} &= \text{ListCost} - \text{ContractedCost} \\ \text{Additional effective effect} &= \text{ContractedCost} - \text{EffectiveCost} \\ \text{Total effective rate effect} &= \text{ListCost} - \text{EffectiveCost}\end{aligned}$$

They may be calculated only on a consistent currency, period, scope and charge population. Purchase and covered-usage records must be filtered according to a documented billed or accrual basis to avoid double counting.

Commitment discounts

Reservations and Savings Plans exchange an obligation to consume or spend over a defined period for reduced rates. Commitment purchases may be upfront, recurring or mixed. Their cost is amortized across charge periods and allocated to eligible usage. Unused commitment is a cost of underutilization, not an additional saving, and follows a use-it-or-lose-it logic.

The prototype contains commitment-related dimensions and cost measures, but it must not claim complete commitment utilization analysis until identifiers, statuses, purchase rows, covered usage and unused portions have been validated in the dataset.

Realized versus potential savings

- A realized rate effect can be derived retrospectively from comparable populated cost measures, with explicit exclusions.
- A potential saving estimates a future action and normally requires additional evidence such as utilization metrics, resource inventory, provider recommendations, price references or commitment coverage.

The current dashboard's `ListCost - EffectiveCost` indicator should therefore be described as an effective difference from list cost until its scope and interpretation are fully validated. It must not be presented as a verified business saving without that validation.

2.4 - Allocation, Showback and Governed Self-Service

Allocation and accountability

Allocation associates technology expenditure with an accountable business perimeter such as a domain, application, project, team or cost center. Unallocated costs must remain visible rather than being silently removed because they are themselves a governance signal.

Showback and chargeback

Showback exposes the cost attributed to a consumer without necessarily performing an internal financial transfer. Chargeback goes further by assigning or recovering that cost through an internal accounting mechanism. The current PFE implements a showback-oriented decision-support scenario; it does not implement an enterprise chargeback process.

Governed self-service

The proposed access model gives managers information for their authorized perimeter, FinOps consolidated visibility and definition ownership, and authorized analysts read access to certified datamarts. Dashboard-level filtering requires corresponding controls on direct data access. The project has not yet validated those controls with separate user identities.

Connection to the professional problem

The project framing identifies repeated requests and manual extracts as possible sources of effort. Their frequency and cost have not been measured. A self-service scenario can test whether users complete specified analyses without a new FinOps extract, but an organizational benefit would require evidence from the actual workflow.

3. Methodology and Evaluation Framework

3.1 - Objectives and Methodological Framework

The method works backward from the intended outcome: stakeholders should be able to examine attributed cloud costs through understandable indicators. The research questions in section 1.3 are translated into capabilities, artifacts and acceptance criteria. The table follows this deduction down to its prerequisite, a suitable dataset; it does not describe the execution schedule or claim that every criterion has been met.

Intended outcome	Required capability	Project artifact	Acceptance criteria
Support FinOps decisions	Integrate analysis and operational monitoring	FinOps data platform	Indicator meanings and data provenance are identifiable
Provide stakeholder showback	Publish understandable, authorized cost views	Dashboard and documented KPIs	Business questions can be answered; totals and access boundaries are checked
Ensure consistent calculations	Define grain, measures and aggregation rules	Analytical model, dictionary and datamarts	Questions map to data; calculations preserve grain and reference totals
Maintain trust as sources evolve	Control quality, responsibilities and changes	Versioned Data Contract and quality reports	Incompatible inputs are detected; invalid publication is blocked and explained
Keep products updated	Automate processing and recovery	Daily/Monthly jobs and audit records	Expected periods are processed; replay avoids duplicates; failures are diagnosable
Establish feasibility	Test the complete chain locally	End-to-end POC	Source, transformed and displayed values agree
Supply experimental inputs	Generate reproducible normal and correction cases	Synthetic FOCUS corpus, generator and manifests	Schema, coverage, totals and scenario expectations are verified

Implementation proceeds from the corpus and local POC to automated cloud processing and online consumption, with quality, governance and security accompanying each stage. Repeated build-and-test cycles are consistent with the FinOps Foundation's FOCUS adoption guidance. The sections below explain this method; architecture, implementation and evaluation are developed in chapters 4, 5, 7 and 8.

3.2 - Experimental Dataset Preparation

3.2.1 Reference data and source preparation

Two Azure FOCUS cost-and-usage files, anonymized before use, provide reference rows and schema for fictional consumption data. Microsoft's FOCUS file-schema documentation defines the fields but does not establish the provenance of these particular files. The reference inputs remain local and are not published with the project.

The independent generator retains rows with usable charge dates and billed cost, EUR currency and Microsoft provider. The `daily-template-v2` method samples these rows, replaces selected identifiers consistently, assigns dates and scales costs to configured targets. Daily output retains the source Arrow schema and is written to `datasets/focus/daily/YYYY/MM/YYYY-MM-DD.parquet`. The same published inputs can subsequently feed different analytical engines.

A date-specific seed makes sampling independent of batch boundaries. The configuration sets an annual `BilledCost` target of EUR 12 million for 2025, monthly weights and a 1.06 growth multiplier for 2026. Targets are distributed across days in cents. These are modeling assumptions, not estimates of observed seasonality or company spending.

A configuration-and-source fingerprint and daily ledger record paths, rows, amounts and SHA-256 hashes. Resumption checks existing entries and files rather than silently replacing them. Stable synthetic identifiers preserve selected links; their substitution alone does not prove that every potentially identifying attribute has been removed.

3.2.2 Monthly billing and controlled scenarios

A monthly close needs a billing reference distinct from provisional Daily usage. Without a real invoice feed, the simulator constructs this reference from a complete month of Daily files. It requires every calendar day, matching ledger entries and hashes, consistent schemas and periods, and agreement on rows, currency, provider and `BilledCost`.

The `no_change` scenario concatenates rows unchanged. Other scenarios inject known differences: `late_usage` adds a EUR 1,500 Usage line; `cost_correction` reduces an eligible line by EUR 200; `combined` applies both; and `unexplained_difference` changes a line by EUR 50 without an explanation. Deterministic selection makes these interventions repeatable.

The manifest records affected rows, before/after amounts and expected differences. It serves as an evaluation oracle for detected changes, missed changes and blocking decisions, but must not inform the pipeline's decision to close a month. Financial acceptance rules require FinOps review; a known injected amount is not automatically an acceptable difference.

Monthly output is published locally at `datasets/focus/monthly/billing-YYYY-MM.parquet`, with a separate manifest.

Read-back checks precede publication; explicit replacement archives the previous version. Daily and Monthly represent the same month's activity, so adding their amounts would double-count it.

3.2.3 Coverage and limitations

Local validation on 27 September 2026 confirmed schema, temporal and monetary consistency for 608 Daily files, January 2025-August 2026, containing 3,664,261 rows and EUR 20,357,040. The 18 `no_change` Monthly files cover January 2025-June 2026, with 3,279,613 rows and EUR 18,220,080; each matched its Daily month and manifest hash. Revalidation after path harmonization on 28 September preserved these results and the Monthly hashes.

July-August therefore belong only to the validated Daily corpus. Corrected scenarios still require downstream tests against expected outcomes. The corpus supports engineering evaluation, not claims about real resource lifecycles, invoice correctness or organizational savings; forecasting would assess behavior under the generator's assumptions.

3.3 - Data Quality, Governance and Change Control

Quality controls cover schema, required values, periods, monetary reconciliation and repeatability. Technical validity is assessed separately from business attribution: a reconciled total can still lack an application or owner. Missing allocation attributes remain visible, and negative amounts are not rejected solely by sign because they may represent credits. EUR and Microsoft delimit this corpus, not all FOCUS inputs.

The versioned Data Contract declares fields, types, nullability and selected rules. Failed validation must block invalid canonical publication and identify the input and rule for diagnosis. Passing this project contract is not certification of complete FOCUS conformance. Governance also requires traceable revisions, corrections and access decisions. The proposed responsibilities assign business definitions to FinOps and technical controls to data engineering; organizational confirmation remains necessary.

Specification and contract versions are explicit because FOCUS version numbers do not guarantee compatibility, as stated in the [official repository](#). The project's compatibility policy permits additional fields, but rejects missing required fields and incompatible type changes. Intentional evolution requires review of field meanings, a versioned adaptation and regression tests of dependent calculations. Schema detection does not automatically revise the contract or detect every semantic change.

Access tests cover files, queries, dashboards and exports. Public synthetic demonstrations and authorized stakeholder services are assessed separately; chapter 7 reports the available security evidence.

3.4 - Progressive Implementation and Automation

The local POC tests agreement between source totals, transformations and displayed indicators for an identified period. The cloud extension then addresses recurring execution, separate development and production outputs, and remote consumption. This explains the implementation strategy without assuming a strict chronology or treating local success as cloud validation.

The operating protocol distinguishes historical backfill, incremental Daily processing and monthly replacement. Each workflow defines input scope, expected outputs and replay behavior. Development validation precedes production promotion; source identity, status and reconciliation results must support recovery if one environment succeeds and the next fails. Repeated execution must not duplicate activity.

Evaluation follows the acceptance criteria in section 3.1. Each test records its inputs, period, environment, revision, procedure, expected result and observed output, including failures. Cloud records also identify runs and relevant table states. Code demonstrates implemented logic; retained outputs establish behavior in a particular environment. Chapter 7 separates available results from unresolved tests.

Performance and cost comparisons require equivalent workloads and verified outputs, with compute, cache and measurement boundaries recorded. Shared and idle intervals need explicit attribution. Otherwise the comparison is an operational observation rather than a controlled benchmark; chapter 8 develops this protocol.

3.5 - Analytical Preparation

Each business question determines a measure, population, period and allocation scope. For example, monthly cost by application requires a stated cost basis and attribution rule. Indicator specifications record source fields, formula, unit, grain, filters, exclusions and missing-value treatment, using the financial definitions in chapter 2.

These specifications guide the dimensional model and subject-oriented datamarts. Joins and aggregations must preserve the intended charge population without multiplying costs. Each datamart is compared with a reference calculation on the same scope; displayed indicators are then checked against the datamart read by the dashboard. Section 4.5 defines the concrete model and maps its products to business questions, while chapter 6 examines their analytical interpretation.

3.6 - FinOps Data Product Delivery and Operation

The dashboard is treated as a stakeholder-facing analytical data product with defined questions, consumers, measures and maintenance expectations. This use of the term follows Google Cloud's definition, which includes dashboards. Datamarts are reusable analytical products; the platform provides their production, validation and distribution.

Delivery requires an identifiable deployed version, verified permissions and visibility of the last successful update, covered periods and known quality limitations. Issues must be traceable to inputs, rules and runs. The PFE's production environment is not equated with enterprise deployment.

Acceptance combines calculation and access checks with defined user tasks, such as identifying a cost driver or examining allocation. Results are compared with reference outputs and representative feedback where available. Technical success alone does not demonstrate stakeholder adoption, reduced effort or realized savings; those claims require separate evidence.

4. General Design and Architecture Choices

4.1 - Design Requirements and Principles

The architecture follows the requirements in section 1.5. It must make source charges traceable, validate data before publication and serve consistent indicators to users with different access rights. The distinction between these requirements and the available tools matters because the local POC, cloud prototype and possible enterprise deployment have different operating conditions.

Functional drivers

- ingest recurring FOCUS cost and usage data;
- preserve received evidence and its provenance;
- validate a versioned data contract before canonical publication;
- publish consistent FinOps data products for Inform, Optimize and Operate;
- support dashboard users and authorized analytical consumers;
- reconcile row counts and financial measures across transformations.

Quality and governance drivers

- explicit ownership and definitions;
- controlled schema evolution;
- record-level and batch-level lineage;
- reproducible processing and validation;
- visibility of rejected, missing or unallocated information;
- separation between canonical data and derived analytical products.

Security drivers

- least-privilege and read-only analytical access;
- isolation of managerial or domain perimeters;
- protection of direct SQL access in addition to dashboard filtering;
- no credentials committed to source control.

Operational drivers

- restart and recovery information;
- observable freshness and execution status;

- maintainability by the platform and FinOps roles;
- portability between the local validation environment and the target cloud environment where feasible.

Project constraints

The design must remain achievable within the PFE schedule and available licences. Confidentiality limits direct use of enterprise billing data. Only a restricted anonymized source is available. The local DuckDB POC, the Databricks/GCP DEV and PROD workspaces, and a possible enterprise Azure target provide different levels of evidence.

4.2 - General Platform Design (Logical HLD)

System boundary

The system begins with cost and usage datasets delivered by a billing-data producer and ends with governed information exposed to FinOps, managers and authorized analysts. Provider billing generation, enterprise accounting, resource remediation and production identity administration remain external responsibilities.

Logical flow

```
Cost and usage sources
      v
Evidence preservation
      v
Ingestion and lineage
      v
Contract and quality validation
      v
Canonical cost and usage data
      v
Dimensional and subject-oriented data products
      v
Governed dashboards and analytical access
```

Quality, reconciliation, security, metadata and observability apply across the complete flow.

Logical components

Component	Responsibility
Source interface	Receive periodic cost and usage data with identifiable origin and period
Evidence store	Preserve the received representation without hidden business transformation
Ingestion service	Register the batch and add technical lineage
Contract gate	Reject or quarantine incompatible structure and invalid mandatory values
Canonical repository	Provide the authoritative standardized FinOps representation
Analytical model	Organize detailed charges by time, scope, resource, service, price and commitment
Data-product layer	Publish stable subjects and common measures for consumption
Consumption layer	Provide role-appropriate dashboards and analytical access
Operational control	Track executions, freshness, failures, reconciliation and recovery

Architectural principles

- The immutable evidence layer and the canonical repository have different responsibilities.
- Validation occurs before data becomes authoritative.
- Derived products do not become independent sources of truth.
- Consumers use stable products rather than storage-specific implementation details.
- Access control applies at every exposed interface.
- Claims of reliability require reproducible evidence, not only an architecture diagram.

Operating model: centralized production and distributed consumption

The proposed operating model gives managers access to their cost scope and authorized analysts access to further analysis. The Data Platform retains shared technical capabilities; FinOps owns business definitions and product validation; domains consume only their authorized scope. These responsibilities describe the target design and require organizational confirmation.

Here, “data factory” denotes a centralized production organization, not the Azure Data Factory service. Centralized production can coexist with autonomous access. The issue under study is dependency on repeated requests for changes.

Data Mesh combines domain ownership, data as a product, self-service infrastructure and federated governance. Its autonomy extends to producing teams, beyond report consumption. Physical data location alone does not define it. Dehghani, 2020.

The target adopts product-oriented practices-definitions, quality, ownership and stable interfaces-without autonomous production by several domains or full federated governance. It is therefore not described as a complete Data Mesh. Chapter 9 will revisit this choice against the evidence collected.

This section describes what the platform must do without assigning those responsibilities to products. Section 4.3 compares the documented technology options and identifies the tools used in the prototypes. Section 4.4 maps the logical flow to deployed components; section 4.5 explains the data model and the analytical products.

4.3 - Technology Evaluation and Selection

4.3.1 Evaluation basis

The project's initial architecture evaluation considered Microsoft Fabric, Azure Databricks and a modular Azure PaaS solution against requirements for ingestion, contract validation, governed analytical products and reporting. It also listed maintainability, security, cost, available licences and feasibility within the PFE as criteria. The evaluation was qualitative: it contained neither weighted scores nor a measured total cost for the three candidates. It can explain which risks were considered, but it cannot support a numerical ranking.

4.3.2 Platform alternatives and implemented scope

Fabric groups data preparation, storage, SQL and Power BI within one service environment. The initial record treats this integration as a possible advantage and flags capacity and licensing as questions. Azure Databricks offers a lakehouse processing path with Unity Catalog and SQL serving, but the proposed Azure architecture would also require storage, identity and reporting integration. A modular Azure solution would make service boundaries explicit while increasing the number of components to configure and operate. These are design observations from the initial evaluation, not benchmark results.

The implementation subsequently converged on Databricks with GCP object storage for the active cloud prototype. Its code and catalog organization are described in sections 4.4 and 5.3; execution evidence is assessed in section 7.4. This implementation choice establishes what was built; the available decision record does not isolate the commercial, access or scheduling reason for the move from the Azure-first options. The thesis therefore does not present the GCP prototype as an approved enterprise Azure deployment or infer that it outperformed Fabric on cost.

4.3.3 Storage and processing choices

The independent generator publishes Parquet files as the shared RAW input. Apache Parquet specifies a column-oriented file format for storage and retrieval. The Databricks pipeline publishes managed Delta tables in Bronze, Silver and Gold, where month-level replacement and table history are required. Delta Lake extends Parquet data files with a transaction log; the two formats therefore serve different roles in this design rather than being mutually exclusive alternatives. A replacement is atomic for an individual Delta table operation, while the entire multi-table pipeline needs audit and recovery logic.

Python and PyArrow support deterministic file generation and schema checks. SQL defines the Gold model and datamarts, and PySpark coordinates their execution in Databricks. YAML records the versioned data contract, while DBML documents the dimensional design. DuckDB serves the frozen local POC; its results are evidence for that environment only.

4.3.4 Reporting decision and remaining comparison

The POC uses Streamlit with DuckDB datamarts, whereas the cloud application is designed to query Databricks through a SQL Warehouse. The existing Power BI dashboard remains the professional reference system. The repository demonstrates local Streamlit execution, but it does not contain a controlled comparison of these reporting options or validated Row-Level Security for the new application. The selection can therefore be defended for the implemented prototype and its reproducible tests; an enterprise adoption recommendation would also require access, performance and operating-cost evidence.

4.3.5 Technology choices by function

The following table summarizes the tools present in the two implementations. The local POC is a separate experiment, not the DEV environment of the cloud platform. DEV and PROD use the same Databricks/GCP architecture with separate business catalogs. The table reports implementation choices and their role; it is not a scored comparison of every tool named during early exploration.

Function	Local POC	Cloud DEV and PROD	Rationale and evidence boundary
Source files	Shared synthetic Parquet files on the local filesystem	The same published Parquet history in GCS, exposed through an external Unity Catalog volume	A common input supports comparison; publication and cloud ingestion need separate checks.

Function	Local POC	Cloud DEV and PROD	Rationale and evidence boundary
Data processing	Python/Pandas and local notebooks	PySpark for ingestion and contract processing; versioned SQL for Gold and datamarts	The cloud path uses distributed Spark execution, but no performance advantage follows without a comparable benchmark.
Processed tables	Local Delta layers and a DuckDB analytical warehouse	Managed Delta tables in Bronze, Silver, Gold and datamarts	Delta supports table-level replacement and history; DuckDB supplies local analytical tests.
Orchestration	Explicit local commands and notebook execution	Databricks Jobs coordinating notebooks and validation tasks	The cloud Jobs encode task dependencies; an automatic schedule does not by itself prove end-to-end operation.
Governance and audit	Local contract checks and test outputs	Unity Catalog namespaces and permissions; <code>finops_ops.audit</code> for operational history	Catalog organization is implemented, while effective access and recovery still require run-level tests.
Analytical access	DuckDB queries over local datamarts	SQL Warehouse queries over PROD datamarts and OPS audit	The Warehouse serves the application; it is not the compute engine used by every batch task.
Visualization	Local Streamlit application	Separate Streamlit Databricks App	The local interface is tested; cloud page behavior and permissions need their own evidence. Power BI remains the existing professional reference system.

The initial evaluation documents Fabric, Azure Databricks and modular Azure PaaS as platform alternatives. It does not provide controlled scores for Colab, Airflow, Tableau or every other tool that could be placed in a category. Adding such products to this table would imply an evaluation that the project did not perform.

4.4 - Selected Physical Architecture

Section 4.2 defines the required functions, and section 4.3 explains the chosen technologies. This section places the implemented components in their actual storage, catalog, processing and access boundaries. It answers where data resides and how the components connect; it does not repeat the technology comparison or detail every table, which is the purpose of section 4.5.

4.4.1 System boundary and implementation tracks

The implementation separates three responsibilities. `FinOps Data Generator` produces a single FOCUS-based synthetic dataset, independently of any analytical engine. `FinOps Data Platform - POC` preserves the earlier DuckDB/Streamlit experiment and its January 2025 local results. `FinOps Cloud Data Platform` is the active Databricks/GCP implementation. This division preserves the value of the local validation without implying that DuckDB and Spark execute identical physical code or that local results validate cloud operation.

The cloud platform receives detailed Daily and monthly Parquet files in a shared RAW namespace, transforms them through Bronze and Silver Delta tables, builds a Gold dimensional model, and publishes subject-oriented datamarts. Its read-only application consumes the datamarts and operational audit through a SQL Warehouse. The architecture is an implemented PFE prototype with planned extensions; it is not a claim of deployment into Technip Energies production systems.

```
Independent generator -> GCS RAW Parquet -> finops_raw.landing
                        └─ DEV -> Bronze -> Silver -> Gold -> datamarts
                        └─ PROD -> Bronze -> Silver -> Gold -> datamarts
DEV and PROD runs -> finops_ops.audit
PROD datamarts + audit -> SQL Warehouse -> App
```

4.4.2 Source authority, catalogs and data lifecycle

`finops_raw.landing` exposes the same GCS objects to both environments. DEV business tables reside in `finops_dev`; PROD business tables reside in `finops_prod`. Each contains Bronze, Silver, Gold and datamart schemas. The separate `finops_ops.audit` catalog has five operational tables and records an explicit `environment` on each row. This arrangement permits independent data publication while maintaining a shared operational history. It also makes an environment filter essential in every audit lookup and write.

The source lifecycle follows financial meaning. Daily files populate an open month provisionally. The detailed monthly billing file becomes authoritative after close and replaces that month's Silver and Gold contents. The two source types are never added together in the fact table. A monthly replacement is therefore a correction of the canonical state rather than another batch of cost on top of Daily activity.

Automatic RAW archival is disabled in the common configuration. The earlier design would have moved processed sources after close, but a DEV close could then remove a file needed for independent PROD processing. Keeping the RAW objects allows both environments to consume the same input and preserves the evidence used for replays. This choice has a storage and lifecycle cost: a future retention rule must identify every consumer and the audit period before moving or deleting any source. The archival module remains in the codebase as an optional capability, not an active lifecycle step.

4.4.3 Compute configuration boundary

The repository separates common processing logic from compute-specific deployment artifacts. `platform/common` contains shared notebooks and SQL. `platform/serverless` and `platform/classic_compute` contain the respective setup and Job adapters. Both scenarios target the same logical RAW-to-datamart flow, while catalog storage configuration, compute lifecycle and billing attribution differ. This separation allows the implementation to change compute without redefining the data contract or analytical grain.

This architecture section records the physical components and their interfaces, not the chronology of workspace decisions or a claim that either compute mode is more economical. Chapter 7 records execution evidence and chapter 8 examines the later location and compute choices as an operating-cost case study.

4.4.4 Analytical and consumption layers

Silver retains the canonical FOCUS representation after contract validation. Gold and the fourteen datamarts are stored separately in each business catalog. Versioned SQL defines and refreshes these products; Python orchestrates execution and supplies validated environment identifiers. Some datamarts read central Silver fields absent from the Gold fact, while others read Gold. Section 4.5 specifies the tables, their grain and the business questions they are designed to address. A full datamart refresh favors simple publication in the current version at the expense of extra work after each monthly replacement.

The frozen local dashboard demonstrated that eleven of fourteen local datamarts support its views. The active Databricks App is a separate implementation designed to read PROD datamarts and OPS audit in read-only mode. Its service principal is intended to receive only the Unity Catalog and Warehouse permissions required for that use. Actual effective permissions, direct SQL access and Row-Level Security require their own tests; interface behavior alone cannot establish them.

4.4.5 Architectural trade-offs and evidence boundary

The design prioritizes a single source history, explicit environment isolation, a canonical Silver layer and reproducible SQL products. It accepts duplicated DEV/PROD data, materialized datamarts and retained RAW objects to make testing, promotion and audit easier. These choices incur storage, refresh and operational costs that chapter 8 must quantify. Chapter 7 must establish which cloud paths ran successfully and how they compare with the POC. The existence of two compute configurations alone cannot establish a cost or latency advantage.

4.5 - Data Architecture and Dimensional Modeling

4.5.1 Canonical record and analytical grain

The data architecture separates three purposes. RAW retains the received Parquet evidence; Bronze adds technical lineage without changing source business values; and Silver contains the FOCUS representation after the versioned contract has accepted it. Gold reorganizes that canonical record for recurring analysis, while datamarts answer defined questions at coarser grains. Neither Gold nor a datamart becomes an independent source of cost truth. The same logical model is instantiated in separate DEV and PROD catalogs; the frozen local POC uses a related analytical design but a different engine.

The Gold fact, `fact_finops_cost_usage`, represents one FOCUS cost or usage charge line. It retains `billing_month` so that an authoritative monthly bill can replace the provisional Daily-fed month without adding both sources together. Its technical key is deterministic, using source lineage and logical row position; dimensional foreign keys connect a charge to its time, scope, resource, service, SKU, location, pricing and charge type. This grain permits reconciliation back to Silver and prevents a monthly summary from obscuring individual adjustments. It does not, by itself, establish the financial meaning of a difference between cost columns.

4.5.2 Gold table inventory and relationships

The cloud SQL defines the following Gold tables. This inventory gives each name and responsibility. A full column-by-column DDL listing is not required to interpret the model in this chapter.

Table	Analytical responsibility
<code>fact_finops_cost_usage</code>	Charge-line measures, quantities, billing month and dimensional keys
<code>dim_date</code>	Reusable calendar attributes for the fact's dates
<code>dim_billing_scope</code>	Billing accounts, subaccounts, customers and cost-center attributes
<code>dim_resource</code>	Resource, resource group and application/owner attributes
<code>dim_service</code>	Service, provider, publisher and reseller attributes
<code>dim_sku</code>	SKU, meter, offer and term attributes
<code>dim_location</code>	Region and availability-zone attributes
<code>dim_pricing</code>	Pricing category, unit and currency attributes
<code>dim_commitment_discount</code>	Commitment identifiers and categories

Table	Analytical responsibility
dim_charge_type	Charge category, subcategory and frequency
dim_tag	Normalized tag key/value pairs
bridge_resource_tag	Many-to-many association between resources and tags

The model has ten dimensions. The fact links directly to nine of them; tags are reached through `dim_resource`, `bridge_resource_tag` and `dim_tag`. The model's billing-scope and resource dimensions currently use Type 1 updates. Their validity columns do not prove that every past attribute value is preserved as SCD2 history. The source dataset lacks a native charge identifier and availability zone, so the implementation generates a deterministic charge identifier and uses `Unknown` for the unavailable zone. These are model limitations, not general properties of FOCUS.

4.5.3 Datamarts and business questions

The datamarts are analytical products derived from the canonical representation. The question column states their intended use, not a measured benefit or a claim that every cloud table has been independently validated. The paragraph after the table distinguishes products derived from Gold from those reading central Silver directly.

Datamart	Business or control question supported
dm_monthly_billing	What is the billed cost for each billing month?
dm_daily_billing	How does billed cost vary by charge-start date?
dm_cost_by_scope_service_month	Which cost centers, customers and services account for monthly billed cost?
dm_top_services	Which service categories and names have the highest billed cost in the loaded data?
dm_top_resources	Which resources have the highest billed cost in the loaded data?
dm_cost_by_charge_type	How is monthly billed cost distributed across charge categories and frequencies?
dm_sku_cost	Which SKUs and meters account for the highest billed cost?
dm_savings_monthly	How do monthly List, Contracted and Effective Cost totals differ?
dm_executive_summary_monthly	What are the monthly cost totals, charge-line count, resource count and service count?

Datamart	Business or control question supported
<code>dm_top_resources_monthly</code>	Which resources drive billed cost within a selected billing month?
<code>dm_data_quality_monthly</code>	How many critical nulls and ingestion batches appear in each Silver month?
<code>dm_cost_by_resource_group_month</code>	What are monthly billed cost and resource count by resource group?
<code>dm_cost_by_subscription_month</code>	What are monthly billed cost and resource count by subaccount or subscription?
<code>dm_cost_by_application_owner_month</code>	How are monthly billed cost and resource count attributed to applications and owners?

`dm_savings_monthly`, `dm_executive_summary_monthly` and `dm_data_quality_monthly` read the central Silver representation because they use FOCUS fields or technical metadata not all held in the Gold fact. The remaining datamarts read Gold. The differences named as “savings” in the SQL are arithmetic cost-column comparisons; they do not establish realized savings without eligibility rules, charge-category analysis and business validation. Section 5.3.3 describes how the cloud model and datamarts are built, while chapters 6 and 7 examine their analytical meaning and validation evidence.

5. Detailed Design and Implementation

5.1 - Independent FOCUS Data Generator

The generator is a separate project that produces one synthetic FOCUS-based source history for the local POC and the cloud platform. This is a logical account of the final system, not a claim that the generator was extracted before the POC was built. Its configuration identifies the anonymized reference data, output and ledger locations, date range, row and cost targets, monthly weights, and identifier salt. Publication to GCS is a separate step; the generator does not write DuckDB or Unity Catalog tables.

5.1.1 Daily data generation

The command-line interface generates the configured history, an explicit date range, the next number of days, or the remainder of the next pending month. For each date, a date-specific deterministic seed selects reference rows. The module assigns charge and billing periods, transforms supported identifiers consistently, scales cost fields, and casts the output to the source Arrow schema. It writes compressed Parquet under `datasets/focus/daily/YYYY/MM/YYYY-MM-DD.parquet`.

The file is written provisionally, hashed with SHA-256, and moved to its published local path. A ledger records its path, row count, target and actual `BilledCost`, and hash. A fingerprint of relevant settings and source hashes detects changes to the generation context. Existing output requires explicit resume behavior; resume verifies recorded files instead of silently regenerating them. A local file lock prevents concurrent generator processes from publishing to the same ledger. These controls support local reproducibility, but do not make a subsequent GCS copy a distributed transaction.

5.1.2 Monthly billing simulation

A monthly bill can be built only after every Daily file for the target month has been found and checked against its ledger entry, schema, currency, provider, date boundaries, and billed-cost total. In the `no_change` scenario, the detailed Daily rows are concatenated without aggregation. Four other scenarios introduce late usage, cost correction, a combination of both, or an unexplained difference. For each intervention, the manifest records the source and resulting billing row, input hashes, scenario, expected cost difference, and output hash. This manifest serves as an experimental oracle for reconciliation; the cloud pipeline does not read it to decide whether a difference is valid.

Monthly output follows `datasets/focus/monthly/billing-YYYY-MM.parquet`, matching the relative path used in GCS. Overwrite is explicit and copies the previous Parquet and manifest to a dated local archive. The replacement file is read back against the intended Arrow table before publication. This local version archive is distinct from the cloud platform's

currently disabled automatic RAW archival. The generator neither closes a cloud billing month nor deletes its source files.

5.1.3 Validation and publication of source files

On 27 September 2026, local validation covered 608 Daily files, 3,664,261 rows and EUR 20,357,040 for January 2025-August 2026. It also covered eighteen `no_change` monthly bills through June 2026, with 3,279,613 rows and EUR 18,220,080. July and August 2026 therefore belong to the validated Daily history, not to this validated monthly-bill set. These figures establish internal consistency of synthetic files, not realistic company spending or successful cloud ingestion. Section 3.2 describes the dataset preparation and validation method.

The same generated Parquet files may feed the local POC and, after a controlled copy, the GCS RAW namespace. The local and cloud processing implementations are described separately in sections 5.2 and 5.3. Section 3.2 discusses the source assumptions and representativeness limit; section 7.4 identifies the cloud run evidence still required.

5.2 - Local End-to-End Proof of Concept

The local POC is a frozen DuckDB/Streamlit experiment. It demonstrates a processing and reporting chain for synthetic January 2025 data without implying that the later Databricks implementation runs the same physical code. Its results are treated as local evidence throughout this thesis; cloud execution is assessed separately in section 7.4.

5.2.1 DuckDB processing chain and data contract

The POC reads detailed monthly Parquet, processes Bronze, Silver and Gold data, and publishes analytical tables into a local DuckDB warehouse. The contract boundary checks the source before canonical Silver publication. This establishes an end-to-end local route from generated bill to queryable output, but it does not validate the behavior of the cloud contract implementation or its operational recovery controls.

For January 2025, the recorded POC outputs contain 164,145 charge lines and EUR 912,000 in `BilledCost` through Bronze, Silver and the Gold fact. The local integrity check reports no orphan foreign keys. Section 7.1 examines these results and their limits.

5.2.2 Gold model and subject-oriented datamarts

The local model preserves the charge-line grain and provides dimensions for recurring cost questions. DuckDB SQL templates publish fourteen materialized datamarts in the `datamart` schema; eleven feed the local dashboard. Subjects include executive cost and volume, cost-basis comparisons, top resources, quality and batch traceability, resource groups, subscriptions, and ownership attribution.

Materialization keeps repeated analytical expressions outside the interface and permits reconciliation with the local source. It also duplicates data and requires coordinated refresh. The local test establishes publication for one synthetic month; it does not establish a measured latency advantage or cloud table-level reconciliation. The cloud project implements corresponding subjects in its own versioned SQL rather than executing the DuckDB templates unchanged.

5.2.3 Local Streamlit dashboard

The POC's Streamlit application reads DuckDB datamarts without writing to them. Its Knowledge Base explains column names and cost formulas alongside the analytical views. Streamlit AppTest exercised eight subviews for January 2025, and the displayed EUR 912,000 billed cost and 164,145 rows reconcile with the local checks. Section 7.1 presents the validation scope. This does not establish effective cloud permissions, Row-Level Security or the behavior of the separate Databricks application.

5.3 - Databricks Cloud Data Platform

The cloud implementation consumes the shared Parquet history but has its own processing code, SQL products and operational controls. It is a PFE platform prototype, not a deployment into Technip Energies' production systems. The technical design below identifies implemented behavior and separates it from cloud outcomes that still need run-level evidence.

5.3.1 RAW, DEV, PROD and OPS organization

The external `finops_raw.landing` volume exposes the same GCS source objects to DEV and PROD. Each business catalog, `finops_dev` and `finops_prod`, has separate Bronze, Silver, Gold and datamart schemas. `finops_ops.audit` keeps operational runs, month status, snapshots, reconciliations and archive events with an explicit `environment` field. Every OPS read and write must preserve that environment distinction.

Bronze retains source values with ingestion metadata, including source-file lineage and run identification. Silver is the canonical validated cost-and-usage representation. Gold provides the charge-line fact and dimensions; fourteen datamarts serve recurring analytical subjects. The central Silver serving table is a copy of the canonical representation, not a second authority. The cloud artifacts are organized under `platform/common`, `platform/serverless` and `platform/classic_compute`; chapter 8 examines what the compute variants mean for cost rather than treating either layout as proof of a cost advantage.

5.3.2 Daily ingestion, backfill and monthly close

The Daily pipeline accepts a source URI and rejects an empty Parquet. It writes Bronze, applies the contract, prepares Silver, requires one open billing month, and refreshes that month in Gold and the datamarts. Source-file lineage prevents a previously ingested path from adding duplicate Bronze or Silver rows. The month remains `OPEN`, and the run identifier links data writes to OPS records. A changed object at an already processed path is not treated as a valid correction; RAW immutability is an operating assumption.

The current Classic Daily Job discovers the oldest RAW Daily file absent from PROD, then processes and validates the same URI in DEV before a promotion gate can admit PROD. Choosing against PROD allows a run interrupted after DEV success to rediscover the file; DEV should then write no additional rows. The template queues runs and limits concurrency to one. It handles one candidate per run, so a backlog requires repeated runs or a sequential catch-up controller. The working completion note reports a first Daily DEV load for 1 July 2026 with PROD promotion disabled; it does not document a full promotion and replay. Section 7.4 records this evidence boundary.

For a closed month, the detailed `billing-YYYY-MM.parquet` file becomes the authoritative source. The close function lands and validates it, checks that it contains the requested month, and records `BEFORE`, `SOURCE` and `AFTER` snapshots. These include row count, monetary sums, account and resource counts, critical-null count, charge-period bounds and, when available, a Delta version. `SOURCE` minus `BEFORE` measures the change from provisional Daily data; it is not automatically an error or saving. The function replaces that month in canonical and serving Silver, verifies `AFTER` row count and `BilledCost` against `SOURCE` within tolerance, requires zero critical nulls, refreshes Gold and datamarts, and sets `CLOSED_DATA_LOADED` under the current non-archiving configuration.

Historical backfill invokes the same monthly-close function sequentially over an inclusive month range. Month-scoped Delta replacement is atomic for each target table operation, but the full sequence across Silver, Gold, datamarts and OPS is not one multi-table transaction. Audit states, reconciliation and rerun procedures are therefore needed after a failure between steps. A dedicated Classic monthly-close DEV-to-PROD Job remains to be constructed and tested. Automatic RAW archival is disabled so that a DEV close cannot remove a bill still needed by PROD; optional archive code is not part of the documented normal close.

5.3.3 Data contract, Gold model and datamarts

Section 4.5 sets out the analytical grain, table inventory and intended business questions. This section identifies the cloud implementation choices and their current evidence boundary.

The versioned FOCUS contract is the gate between Bronze and Silver. It defines expected types, required fields, nullability and selected quality rules. Extra source fields are permitted,

while a missing required field or incompatible type is rejected. Project-specific rules require EUR billing currency and Microsoft as provider, ordered billing and charge period boundaries, and non-null `BilledCost`. Negative billed cost remains valid for credits and adjustments. Pricing and consumed quantities and units are present but optional. These restrictions describe the Azure-derived synthetic corpus, not FOCUS as a universal provider rule.

One remediation is deliberately narrow: an `Adjustment` with null `ServiceName` may use non-empty `ChargeDescription`; unrelated null service names remain invalid. Contract failure prevents Silver publication for that input. Local code and tests document the rule; its exact Databricks execution outcomes still require the run records described in section 7.4.

Gold centers on `fact_finops_cost_usage` at billing-line grain, with ten dimensions for date, billing scope, resource, service, SKU, location, pricing, commitment discounts, charge type and tags, plus a resource/tag bridge. Billing-scope and resource attributes currently receive Type 1 updates; validity columns do not yet prove complete SCD2 history. Versioned cloud SQL defines and loads these structures, while Python passes validated environment identifiers and orchestrates execution. Monthly billing replaces, rather than adds to, the Daily-fed month. The cloud project defines fourteen subject-oriented datamarts in versioned SQL under `platform/common/sql/datamarts/table_refresh`. Some read Gold; others use central Silver fields not present in the fact. Full refresh simplifies publication but repeats work after each source change. The local POC validated fourteen DuckDB datamarts; that result cannot substitute for table-level DEV/PROD output and reconciliation evidence. The Belgium completion plan reports a successful historical backfill for January 2025-June 2026, but the executed commit, Job and Run IDs, control outputs, row and cost totals, and OPS snapshots must be indexed before the thesis treats it as an independently validated cloud result.

5.3.4 Jobs, audit, validation and recovery

Jobs coordinate environment checks, source selection, loading, promotion and validation. OPS records run status and monthly snapshots independently of the business catalogs. The snapshot sequence distinguishes a source correction from a failed publication: reconciliation requires AFTER to match validated SOURCE, while an interrupted run may leave some downstream tables older than Silver. Recovery therefore starts by locating the run, environment, month, source URI, Delta versions and failed task, then reruns the scoped operation and its controls. A green Job status alone is insufficient evidence of cross-table consistency.

The working completion note reports DEV and PROD backfill controls, but primary run artefacts remain to be assembled. A complete Daily promotion, an idempotent replay, a Classic monthly-close Job and an interrupted-run recovery test need separate records. Section 7.4 defines the validation protocol and current evidence boundary.

5.3.5 Streamlit application and read-only access

The separate cloud application under `FinOps Cloud Data Platform/apps/finops_dashboard` is designed to query `finops_prod.datamart` and `finops_ops.audit` through a SQL Warehouse without writing to Unity Catalog. Its pages cover the Knowledge Base, executive overview, cost drivers, savings, allocation and accountability, resources, operations and quality, and architecture. The Knowledge Base explains column names and cost calculations; comparisons of `ListCost`, `ContractedCost`, `EffectiveCost` and `BilledCost` remain descriptive until business eligibility and exclusion rules are validated.

The deployment guide specifies narrow read permissions for the application service principal. Source code and prescribed grants do not establish effective access: deployed page behavior, Warehouse permissions, direct SQL access and any Row-Level Security claim require tests and screenshots. The local Streamlit AppTest result in section 5.2 is not evidence for this application.

6. FinOps Analysis of Results

6.1 - Scope and Method for FinOps KPI Analysis

The analytical question is whether the published products make cost and usage information interpretable for FinOps decisions. Building a dashboard is insufficient evidence by itself: a displayed figure must be tied to a named measure, grain, period, filter, source table and reconciliation rule. This chapter therefore uses the same definition for an indicator when it appears in several views and checks that subject-oriented datamarts agree on shared totals. Section 7.1 reports the local technical checks; this chapter examines what their values can and cannot mean to a business user.

The quantified dashboard observations currently documented come from January 2025 in the frozen DuckDB POC: 164,145 charge lines, EUR 912,000 in billed cost and fourteen published datamarts, eleven consumed by Streamlit. Section 7.1 presents the local validation scope. The generator has a longer synthetic Daily history, and the Belgium working note reports an eighteen-month DEV/PROD billing backfill. Those facts define potential analysis coverage, but local January indicators cannot be combined with cloud monthly indicators until the cloud outputs, executed revision and reconciliation evidence are recorded. An eighteen-month synthetic trend would illustrate the analytical workflow, not the real evolution of the host organization's expenditure.

For each KPI, the analysis records its business purpose, FOCUS input fields, formula, unit, charge-line or aggregate grain, period, exclusions, source datamart and interpretation. `BilledCost` describes the billed amount represented by the source; `ListCost`, `ContractedCost` and `EffectiveCost` express different price perspectives. Arithmetic differences among them are comparisons, not automatically realized savings. In particular, eligibility, charge categories, credits, commitments and scope must be reviewed before labeling a difference as an optimization result.

The dashboard subjects are executive overview, price comparisons, allocation, resource drivers and data quality. An overview should reconcile to the same billed total as the monthly billing datamart. Allocation slices should reconcile to that total after explicit treatment of unknown or unassigned values. Resource rankings identify where to investigate; without utilization, criticality and business context, high cost alone does not identify waste. Data-quality indicators and audit freshness determine whether a business user should trust a period's analysis. The operational and architecture pages explain lineage and status but are not substitutes for validated cost KPIs.

The local POC supports an Inform demonstration and structured investigation of possible Optimize questions. It has not established realized company savings, automated remediation,

identity-based filtering or sustained Operate outcomes. The distinction between demonstrated behavior and intended use governs the interpretations in sections 6.2 and 6.3.

6.2 - Analysis of Costs and Pricing Mechanisms

The executive datamart reports EUR 912,000.00 in `BilledCost` for 164,145 charge lines. The corresponding `ListCost` is EUR 1,290,989.95, `ContractedCost` EUR 1,044,652.85 and `EffectiveCost` EUR 1,101,755.75. Section 7.1 summarizes this local POC validation. Billed and effective cost have different FOCUS meanings and therefore should not be expected to coincide simply because both appear in a dashboard. Their divergence requires interpretation against charge categories and the synthetic generator's transformations before it can inform a financial decision.

The list-to-effective difference is EUR 189,234.20, or 14.66% of list cost. This describes a price-metric gap within the synthetic dataset. It is not a saving produced by the platform, nor evidence that the organization could recover this amount. The contracted-to-effective difference is negative EUR 57,102.90. Describing it as positive commitment savings would reverse the observed sign and misrepresent the data. Before using either gap for optimization, the analysis must align charge eligibility, credits, negotiated rates, commitment-covered usage, amortization and aggregation grain. In the present dataset, the safe label is “cost comparison,” not “realized savings.”

The same local datamart allows cost-driver analysis. Virtual Machines account for approximately EUR 195,399.16 and Storage Accounts for EUR 176,985.27. Together, their unrounded source values represent EUR 372,384.42, or 40.83% of the month's billed total. This concentration gives an analyst a sensible starting point for investigation, but does not establish that these services are overprovisioned or that their cost can be reduced. Utilization, commitments, workload criticality and owner context would be needed for an actionable recommendation.

These results demonstrate that the published subjects can reconcile common measures and expose nontrivial economic questions. They do not validate a company-specific rate policy, a cross-month trend or a monetary benefit. Extending the analysis to the eighteen cloud billing months requires datamart outputs and their Databricks run evidence, rather than extrapolation from January's local POC.

6.3 - Allocation and Resource Analysis

The allocation datamart groups January cost by cost center and service. Its groups reconcile to the EUR 912,000 billed total. However, EUR 875,320.02, or 95.98%, falls under cost center `Unknown`. Only EUR 36,679.98 is assigned to the three named synthetic centers in this view. The result shows limited attribution coverage: a center-level showback can display the data, but it cannot fairly allocate most of the amount to named budget owners. Improving the tagging or mapping rule would have greater immediate value than refining the chart used to present these groups.

Application ownership has a different coverage pattern. Rows with a missing or placeholder `application_owner_id` represent EUR 146,649.15, or 16.08% of the same total. The cost-center and application-owner fields describe different organizational dimensions, so these percentages are not interchangeable or additive. An application owner can be present while the cost center is unknown. Neither field proves that a person has accepted budget responsibility; ownership definitions and access rights still require business validation.

The local executive summary counts 22,453 active resources and 59 consumed services. The most expensive services are Virtual Machines and Storage Accounts, which together account for 40.83% of billed cost as discussed in section 6.2. Resource rankings can help prioritize investigation, but a high charge does not by itself indicate waste. A resource's function, utilization, resilience requirement and commitment coverage must be checked before suggesting a change.

These figures show why data quality and allocation are part of FinOps analysis rather than only pipeline checks. A technically reconciled EUR 912,000 total can still be weak evidence for accountability when most cost-center values are unknown. The finding is specific to the generated January dataset and must not be presented as a diagnosis of Technip Energies' actual tagging or allocation practice.

7. Technical Validation and Performance

7.1 - Local Datamart and Dashboard Validation

This section summarizes local validation of the January 2025 synthetic dataset and the DuckDB and Streamlit prototype. The results apply only to this setting.

Test environment

- dataset: January 2025 monthly billing;
- charge lines: 164,145;
- local analytical engine: DuckDB 1.5.5;
- dashboard framework: Streamlit 1.63.0;
- serving objects: 14 materialized SQL datamarts, including 11 queried by the dashboard.

Reconciliation results

The executive datamart and the existing monthly-billing datamart both report a billed cost of EUR 912,000.00. The executive charge-line count and the monthly quality row count both equal 164,145. The quality table reports zero null values for BilledCost, BillingCurrency and ServiceName, and one ingestion batch for the selected month.

Application validation

Streamlit AppTest executed the initial FinOps page, four Cost Analysis views and four Data & Architecture views. All eight navigable subviews completed with zero application exceptions and zero Streamlit warning elements. Python compilation completed successfully. The existing automated suites also completed successfully: one synthetic-generator test and nine billing-notebook regression tests.

Cost-comparison observations

Indicator	January 2025 result
List cost	EUR 1,290,989.95
Contracted cost	EUR 1,044,652.85
Effective cost	EUR 1,101,755.75
List minus contracted	EUR 246,337.10
Contracted minus effective	EUR -57,102.90
List minus effective	EUR 189,234.20
List-to-effective difference rate	14.66%

The negative contracted-to-effective difference prevents this component from being accepted as demonstrated commitment savings. The result is retained as an analytical observation requiring semantic validation, not corrected or hidden to produce a favorable KPI.

Validity boundary

These results demonstrate local technical execution and reconciliation for one synthetic month. They do not demonstrate Databricks SQL compatibility, production performance, enterprise certification, Row-Level Security, actual organizational savings or forecasting capability.

7.2 - Existing Power BI Diagnosis and Measurement Protocol

The existing Power BI report provides the professional comparison point for reporting needs. The student's account identifies a multi-step change process, but it contains no measured report latency, refresh duration or capacity data. A slow interaction could arise from the semantic model, its source, a visual, network transfer, shared capacity or cache state. The present evidence does not identify which component dominates.

A diagnosis would fix a representative report page, filter setting, user task and expected answer before timing. Initial loading and repeated interactions should be recorded separately because cache state can change the result. Within authorized access, [Power BI Performance Analyzer](#) can separate query, visual display and other elapsed time; model and source inspection would then test a specific cause rather than assume one. Each result needs the report and model version, data volume, connection mode, capacity, hardware, date and number of repetitions. Median and dispersion are more informative than a single selected run.

No such measurements are currently indexed. The comparison in section 7.3 therefore treats the existing report as an operational reference and does not claim that the local Streamlit

prototype improves its response time. Refresh delay and interactive latency would also need separate measurements because they affect different user tasks.

7.3 - Architecture Comparison and Performance Levers

The local POC publishes fourteen materialized datamarts, eleven of which its Streamlit application queries. Section 7.1 establishes their functional execution and reconciliation for January 2025. It does not measure the latency benefit of materialization. An equivalent direct query and datamart query would be needed in the same environment, with identical answers and recorded cache state, to attribute a difference to this design choice.

Cross-system comparisons require further controls. The company's Power BI report and the synthetic local POC differ in data, engine, model, network and capacity. A raw timing difference between them would describe those particular settings rather than establish architectural superiority. Where access permits, query execution, application processing, rendering and end-to-end response should be recorded separately. Storage and refresh cost also belong in the comparison because a materialized answer may be faster to read but more expensive to maintain.

The repository defines Serverless and Classic Databricks scenarios with common processing logic. The earlier Serverless `STANDARD` setting and the Belgium Classic templates differ in compute and Job policy; the reported historical backfill does not resolve those differences experimentally. A controlled comparison would process the same source months and code revision into isolated outputs, verify result equality, and repeat runs with region, runtime, Photon, worker settings, concurrency and cache recorded. Startup time, total duration, DBUs and full provider cost would then be compared. No quantified Serverless or Classic advantage is currently supported by the project evidence.

7.4 - Databricks/GCP Validation

7.4.1 Validation boundary

Cloud validation addresses a different claim from the local DuckDB POC: whether the shared Parquet source can be processed by Databricks, isolated into DEV and PROD catalogs, reconciled, and exposed as governed analytical products. The tests should identify the workspace, region, compute type, Photon setting, executed Git commit, source period, Job and Run IDs, and final table versions. A repository file or deployment guide establishes the intended procedure; an execution record establishes what ran. The available plan locates the work in the Belgium workspace and identifies a Classic scenario, but its summary alone does not provide these run-level details.

The acceptance criteria follow the data path. The RAW file must be readable; Bronze must preserve its rows and source lineage; contract failures must prevent invalid Silver publication;

the authoritative billing month must replace rather than add to provisional Daily rows; Gold and datamarts must reconcile to Silver; and OPS must retain distinct DEV and PROD histories. The application must use permitted read access without write privileges. These criteria test both functional correctness and governance rather than interpreting a green Job icon as complete validation.

7.4.2 Evidence currently available

The working Belgium completion note states that the four catalogs exist, that a monthly backfill for January 2025-June 2026 and DEV/PROD controls succeeded, and that the first 1 July 2026 Daily file was loaded in DEV with `promote_to_prod=false`. Section 3.2.3 independently reports local validation of eighteen `no_change` billing files through June 2026 and Daily files through August 2026. These source counts can be compared with the cloud tables once the relevant Run IDs and query outputs are captured. The cloud claims remain reported execution awaiting primary artefacts, rather than independently reproduced thesis measurements.

Capability	Current record	Evidence needed for a validated thesis claim
Historical DEV/PROD backfill	Reported successful in the Belgium plan	Executed commit; Job/Run IDs; dates; task outputs; rows and <code>BilledCost</code> by month; reconciliation status
Daily DEV load	1 July 2026 reported in DEV only	Run ID; source URI; rows written; DEV checks and month status
Daily promotion to PROD	Job template and procedure exist	Complete DAG run; matching DEV/PROD outputs; OPS records; idempotent replay
Monthly close DEV → PROD	Module and notebook exist; dedicated Classic Job absent	Billing-arrival guard; both runs; <code>BEFORE/SOURCE/AFTER</code> ; equality and failure recovery
Databricks App	Source and deployment guide exist	Deployment record; page checks; Warehouse and Unity Catalog permissions

7.4.3 Daily-to-close experiment

A complete Daily experiment should begin with a RAW file in an open month. The discovery step must choose the oldest file absent from PROD, then DEV load and validation must succeed before the promotion gate admits PROD. The same URI must reach both environments. A deliberate replay of that file should write no duplicate Bronze or Silver rows while keeping the checks green. A no-candidate run should end without loading data. Because the Job processes one candidate per run, a backlog requires repeated sequential runs or an explicit catch-up controller; schedule frequency alone does not clear many pending days quickly.

The monthly-close experiment should begin only when a detailed billing file is present. It must record the provisional BEFORE state, validated SOURCE and stored AFTER state for the same month and environment. The technical reconciliation passes only when AFTER row count matches SOURCE, their BilledCost difference is within tolerance and critical-null count is zero. This demonstrates faithful replacement of the billing candidate; it does not determine whether a nonzero SOURCE-minus-BEFORE difference is financially justified. DEV should pass before PROD is promoted, and a failed or interrupted promotion should be replayable without changing a previously closed month incorrectly. The dedicated Classic orchestration and its test results are still pending.

7.4.4 Limits of current validation

The currently indexed local tests demonstrate generator consistency and cloud-code behavior outside Databricks. The Belgium plan provides a credible progress record for historical cloud loading, but without primary artifacts the thesis cannot report exact cloud row counts, durations, DBUs or application behavior as measured results. No controlled Serverless/Classic performance or cost comparison follows from the presence of both configurations. RLS and direct SQL-access rules require separate access tests. These limits are carried into chapters 8 and 9 rather than filled with estimates presented as observations.

8. Economic Analysis of the Solution

8.1 - Costing Scope and Method

This chapter estimates the cost of building and operating the FinOps platform. Its economic boundary differs from the cloud expenditure analyzed in chapter 6: a charge represented in a FOCUS file is an analytical input, whereas Databricks, GCP services and human work are resources used to produce the analysis. A cost estimate must identify its period, currency, environment and unit of work before figures from different settings can be compared.

Build cost covers design, implementation, tests, documentation and initial deployment. Operating cost covers recurring processing, storage, SQL queries, application use, monitoring and support. Human effort can be valued as recorded person-days by activity and role multiplied by an authorized loaded daily rate. Platform use can be valued from metered quantities and dated prices, with credits, discounts, taxes and any fixed charge shown separately. A shared resource requires an explicit allocation rule; otherwise its cost remains shared.

No complete effort log, approved labor rate or full Databricks and GCP billing extract is currently indexed for this project. Sections 8.2 and 8.3 therefore define the inputs and boundaries of a reproducible estimate. They do not report a project budget or return on investment. A scenario may vary volume, frequency or staffing, but its assumptions must remain separate from observed consumption.

8.2 - Build-Cost Scenarios

The build estimate groups effort by framing and architecture, dataset preparation, pipeline and contract implementation, analytical modeling, application development, security, testing and documentation. For each activity, the relevant record is the time actually spent by role and the scope of the resulting deliverable. Design and learning time should be visible as such; treating all PFE time as industrial delivery effort would obscure the difference between an academic prototype and a staffed deployment.

Cloud resources used during construction belong in the same build boundary when they can be attributed to development or testing. Their valuation needs metered compute, storage and query consumption with the applicable dated price. Credits may lower cash paid during the PFE, but the underlying usage remains relevant to a later deployment scenario. Resources that also serve routine operation must be allocated by a documented rule to avoid counting the same charge twice.

Three scopes can be compared once their inputs exist: the completed local prototype, the Databricks/GCP pilot and a prospective enterprise deployment. The last is a projection and would need assumptions about permissions, support, integration and procurement that are not

established by the prototype. No numerical build total is presented because the effort log, authorized rates and complete platform charges have not yet been assembled.

The organizational options in section 9.2 require a separate comparison at equivalent request volume and quality. External delivery, a dedicated FinOps Data Engineer and a mixed arrangement each involve implementation, review, handoff and possible rework. The student's report of delays identifies a question for measurement; it does not supply the time or cost data needed to rank these options.

8.3 - Operating-Cost Scenarios

8.3.1 Cost boundary and unit of analysis

The relevant economic object is the cost of operating the FinOps platform, not the cloud charges that the platform analyzes in chapter 6. A monthly estimate should include recurring ingestion and close runs, SQL Warehouse use, App hosting where billed, RAW and managed Delta storage, network charges where applicable, monitoring and human support. A run-level view is also needed to compare compute alternatives. The reference unit should be stated explicitly—for example, one historical billing month processed, one newly arrived Daily file, or one calendar month of service—because these workloads have different frequency and data volume.

Let n_i be the number of runs of workload type i , C_i their measured cost per comparable run, and C_{shared} the recurring cost not assigned to a run. The monthly estimate is $\sum(n_i \times C_i) + C_{shared}$. The shared term includes SQL Warehouse and application use, storage, attributable network and monitoring charges, and human support under a stated allocation rule.

This expression defines a method; no numerical result follows from it without consumption data. Build effort from section 8.2 is added only when computing total cost over a defined horizon. Credits or trial allowances affect cash paid, while usage before credits remains relevant to a future operating model.

8.3.2 Measuring Databricks and GCP consumption

The project contains a monitoring SQL script that links Job runs to their task timeline and Databricks billing records. For Serverless or dedicated Job Compute, a Job ID and Run ID can identify DBU usage and list-price cost directly when those metadata are present. Billing-system records may arrive after a run; an empty immediate query result must not be interpreted as zero usage. Report both consumed DBUs and the tariff/currency used for a monetary conversion.

The Belgium Classic templates use an All-Purpose cluster. Its billing records identify the cluster but may not identify the Job run. The monitoring script therefore estimates each run's Databricks share by the time overlap between the run window and cluster billing intervals.

This attribution is reliable only if other notebooks and Jobs did not share the cluster during that window; otherwise their use is mixed into the estimate. The cluster ID, run interval, competing activity and allocation rule must be recorded. Classic total cost also includes the relevant GCP Compute Engine driver and worker charges, disks and network from GCP Cloud Billing. Databricks list-price DBUs alone understate Classic cost. For Serverless, the DBU charge already includes virtual-machine cost, so adding an equivalent VM charge would double count it (Databricks cost guidance).

The RAW retention choice (section 4.4) also has a measurable consequence. Keeping one shared source for DEV and PROD avoids premature removal and supports replay, but GCS storage persists until a coordinated retention rule is introduced. DEV and PROD managed Delta tables and materialized datamarts create additional copies. Their volume and storage class should be measured rather than estimated solely from source-file size.

8.3.3 Scenario comparison

Three operating scenarios can be evaluated with the same model: a supervised PFE demonstration with manual runs, a pilot with scheduled Daily and monthly processing, and wider use with more users or more frequent queries. For each scenario, specify input volume, period, run frequency, concurrency, SQL Warehouse uptime, retention, App usage and human support. Sensitivity analysis should vary these drivers one at a time, particularly Daily backlog, monthly refresh frequency and Classic cluster idle time.

A Serverless-versus-Classic comparison requires the same source months, equivalent transformations and verified output equality, followed by repeated runs with recorded compute settings. Measure startup, total elapsed time, DBUs and full provider cost per processed month or million charge lines. The earlier Serverless `STANDARD` setting with `max_concurrent_runs = 4` was a development configuration; the Belgium Classic Daily Job instead limits concurrency to one. Comparing their unadjusted bills would conflate both compute mode and workload policy.

8.3.4 Current result and interpretation

The repository supplies the costing procedure and monitoring query, but the evidence register does not yet contain the Job and Run IDs, priced DBU outputs or GCP Cloud Billing extract needed for a numeric comparison. The current result is a defined measurement method and cost boundary. A comparison will require observed consumption normalized to a common workload, raw query outputs and dated price sources. A claim about return on investment would also need a comparable baseline and measured business benefit. Section 8.4 uses this method to assess possible changes to Databricks workloads.

8.4 - Applying FinOps to the FinOps Platform

8.4.1 Applying FinOps to the platform itself

The platform exists to improve the interpretation of cloud expenditure, but its own processing and reporting generate expenditure. This creates a second FinOps scope: the cost data being analyzed are the business object, while Databricks, GCP storage and compute are the cost of producing the analysis. Optimizing the latter should not weaken the former. A cheaper run that omits contract checks, delays monthly close beyond an agreed deadline or produces inconsistent datamarts would not be an acceptable improvement.

The first step is attribution. For each workload-historical backfill, one Daily file, monthly close, datamart refresh and dashboard query-record the source volume, Job or Warehouse identity, elapsed time, DBUs, applicable Databricks charge, GCP infrastructure charge and output quality. Useful unit measures are cost per processed billing month, cost per million charge lines and cost per successful Daily file, always paired with processing time and reconciliation status. These measures allow workload growth to be separated from a change in unit efficiency.

Databricks provides `system.billing.usage`, historical list prices and Job timeline tables for this analysis. Billing records may arrive after the run, so data freshness must be checked before treating a missing usage row as zero cost. The standard Job-cost queries do not attribute Jobs that run on an All-Purpose cluster as ordinary Job Compute; the project's Belgium Classic run therefore needs the cluster/time-overlap estimate described in section 8.3 and the associated GCP bill. The method and its attribution uncertainty must be stated alongside any comparison. Databricks billable-usage reference; Databricks Job-cost monitoring.

8.4.2 Workspace location and compute migration

According to the project owner's account, the platform was first exercised in a Frankfurt Databricks workspace with Serverless compute. After the team examined the platform's own consumption, the execution environment was rebuilt in a Belgium workspace using Classic compute. The current repository documents the latter configuration, but the earlier workspace and billing records still need to be archived for a fully traceable chronology. The available records do not establish the monetary saving attributable to location, compute mode or sizing individually. Earlier run identifiers, dates, SKU charges and GCP infrastructure charges must be retained before a numerical before-and-after claim is made.

The migration preserved the logical contract: the same external GCS RAW source and the catalog names `finops_raw`, `finops_dev`, `finops_prod` and `finops_ops`. The Classic deployment added a separate GCS bucket for Unity Catalog managed Delta storage and explicit catalog `MANAGED LOCATION` clauses, while the source bucket remained external and unchanged. A catalog's managed storage location is not the external FOCUS path read through

`finops_raw.landing`. The storage change affected platform setup and compute-specific Job adapters, while common transformations and analytical products remained shared. The working completion note reports historical loading in the new workspace, subject to the evidence limits in section 7.4.

This migration is therefore a case of applying FinOps to the data platform itself, but it is not yet a controlled experiment. Region, Serverless versus Classic pricing, cluster size and lifecycle, Photon, concurrent-run policy and the timing of the runs may all affect the observed bill. Section 8.4.4 specifies how to separate these factors in a comparison.

8.4.3 Optimization levers relevant to this project

Lever	Project-specific question	Measurement and trade-off
Compute choice	Should repeated batch Jobs use Serverless, dedicated Classic Jobs Compute or the current Classic All-Purpose cluster?	Compare full cost and duration for identical output; include GCP VM/disk cost for Classic and idle time for All-Purpose.
Cluster lifecycle	How much paid time occurs before, between and after batch tasks?	Measure idle intervals and evaluate auto-termination or task-level Jobs Compute without missing the next processing window.
Sizing and Photon	Does a larger or Photon-enabled run reduce total cost per successful month, despite a different rate?	Repeat the same workload with one setting changed, record DBUs, GCP resources, elapsed time and output equality.
Job schedule and concurrency	Does processing one Daily file per run cause repeated startup or a slow backlog?	Compare controlled sequential catch-up with the current single-file Job; protect month-level writes and track freshness.
SQL Warehouse and datamarts	Are Warehouse uptime and full refresh of fourteen datamarts justified by dashboard use?	Measure query latency, refresh time, Warehouse active time and table freshness before changing auto-stop or refresh scope.
Data retention	How much do shared RAW objects and DEV/PROD Delta copies cost?	Measure stored bytes and access needs; retain sources until every consumer and audit requirement is satisfied.

These are hypotheses, not universal prescriptions. Databricks recommends matching compute to workload, using Jobs Compute for automated workloads where suitable, terminating idle Classic resources, and testing whether Photon improves cost per workload rather than duration alone. The project's current All-Purpose Classic configuration is a useful development and

migration environment but merits measurement against alternatives once the pipeline is stable. Databricks compute selection; Databricks cost-optimization practices.

8.4.4 Controlled comparison and decision rule

An optimization claim requires an A/B experiment on the same billing files and code revision, with isolated destination tables and identical validation queries. Record region, runtime, cluster or Serverless mode, Photon, worker configuration, concurrency, Warehouse settings, cache state and run order. Repeat runs so that one unusually slow startup or temporary cloud condition does not determine the conclusion. The two Classic backfill templates labeled “with Photon” and “without Photon” do not alone prove a Photon effect if they also differ by environment or input; the setting must be isolated in a comparable test.

For each candidate, compare total provider cost, not only DBUs: Databricks consumption plus GCP Compute Engine, disks, network and storage where applicable, while avoiding double counting infrastructure already included in a Serverless SKU. Then compare cost with elapsed time, data freshness, job failure rate, and equality of Silver, Gold and datamart outputs. A candidate is preferable only if it meets the required quality and operational constraints and improves the chosen cost-performance measure. If the business requires faster availability, a more expensive but much faster configuration may still be justified; the service objective should be stated rather than silently assumed.

8.4.5 Current conclusion

The project already exposes several concrete opportunities: an All-Purpose Classic cluster may incur idle cost; the Daily Job processes one candidate per run; DEV and PROD retain separate Delta products; and the first cloud version refreshes all datamarts. At present, however, the evidence register lacks the priced run-level and GCP records needed to rank these levers or quantify a saving. The immediate FinOps action is to establish the measured baseline in section 8.3, then test one change at a time under the protocol in section 7.3. Chapter 9 will distinguish a measured recommendation from an untested deployment option.

9. Discussion, Limitations and Recommendations

9.1 - Discussion, Limitations and Recommendations

The local validation establishes that the synthetic January 2025 data can pass the implemented processing and reporting path with reconciled row counts and billed cost. It also exposes a limitation of the analytical product: 95.98% of billed cost in that month is assigned to an `Unknown` cost center (Section 6.3). A reconciled total therefore does not, by itself, provide usable accountability. The allocation rules and source attributes need business review before the same measures can support managerial showback.

The project implements a versioned contract, a canonical Silver representation, a Gold model and subject-oriented datamarts. The independent generator makes the input reproducible. These properties support the reliability question in section 1.3 for the local tests. The Belgium plan reports a historical Databricks backfill, but its run identifiers and control outputs are not yet part of the thesis evidence package. A complete Daily promotion, monthly close and recovery test would address claims that the current local results cannot establish.

The governed self-service question remains less resolved. The local Streamlit application executes against datamarts, but the intended manager and analyst identities have not been tested across dashboard, export and direct SQL access. The existing Power BI dashboard already serves several manager groups according to the student's account. A future user study must therefore compare defined tasks and change requests, rather than assume that introducing another interface creates autonomy. Security and business approval remain conditions for any wider access.

Cost comparisons in chapter 6 are descriptive results from generated data. They do not show avoidable expenditure or savings at the host organization. Similarly, chapter 8 defines the platform cost boundary without a complete DBU, GCP and labor-cost dataset. The first economic recommendation is to record comparable workload costs and outputs; only then can Serverless, Classic, scheduling or retention changes be ranked. Performance tests must control input, revision, cache and compute configuration as set out in section 7.3.

The evidence has limited external validity because the detailed history is synthetic and the validated local dashboard covers one month. The construction demonstrates that the engineering method is testable, while actual billing semantics, utilization, user tasks and organizational benefit remain unmeasured. The next validation priority is to capture cloud run artifacts and access tests, then evaluate representative business scenarios on authorized data.

Centralized production and domain use

The proposed model keeps shared ingestion, definitions and controls with the platform and FinOps functions while distributing access to approved products. This can support independent consumption without assigning production responsibility to each domain. The project has not tested whether such an arrangement shortens change requests. Section 9.2 examines an embedded Data Engineer as one possible response to the reported handoffs; its staffing cost and effect require measurement. The local single-domain prototype gives no evidence for adopting a full Data Mesh operating model.

9.2 - Potential Contribution of a Data Engineer Embedded in FinOps

9.2.1 From observation to hypothesis

The workflow in section 1.2 separates management feedback, relay by the manager and implementation by an external contributor. The issue extends beyond dashboard access to FinOps' ability to change data, indicators and reporting directly.

A Data Engineer close to the FinOps business team could bring requirement interpretation nearer to implementation. This proposal does not remove supervisor approval or automatically replace the Data Platform and Power BI teams.

9.2.2 Capabilities and potential benefits to assess

Dimension	Potential contribution	Required evidence or condition
Responsiveness	More direct translation of business feedback into a testable change	Compare end-to-end delay, waiting and iterations
Data handling	Adapt pipelines, contracts, datamarts and aggregates within FinOps	Authorized skills, permissions and technical scope
Business consistency	Validate cost definitions and investigate inconsistent KPIs with FinOps	Documented business reviews, tests and reconciliation
Knowledge retention	Retain context, decisions and knowledge within the team	Documentation, version control, code review and continuity
Human cost	Possible reduction in coordination time or some external work	Total-cost comparison at equivalent scope and quality, including the embedded role

Changing visuals, DAX measures or Power BI models may require specialist BI skills. The Data Engineer title alone does not guarantee that capability; responsibilities with the Power BI specialist must be explicit.

9.2.3 Conditions and limits

The target model could retain a shared platform and standards, an embedded or dedicated FinOps Data Engineer for data products, and Power BI collaboration for reporting. Greater business proximity is not evidence of a Data Mesh implementation.

Changes remain subject to access controls, tests, business acceptance and publication rules. Peer review and written handover reduce dependence on one person.

An embedded role adds cost and may not justify a full-time position given the request volume. Compare a part-time dedicated resource and a mixed model as well. External delivery may remain valuable for specialized or occasional needs.

9.2.4 Proposed evaluation

For a small set of representative authorized requests, record request, approval, handoff, work start, delivery and acceptance dates, plus active time by role and iteration count. Separate elapsed delay from human effort.

Compare observations with a clearly defined embedded scenario, or with a measured pilot if one exists. A simulation does not establish actual company improvement. Chapter 8 will value effort, external services and recurring costs without assuming rates or savings.

10. General Conclusion and Outlook

10.1 - General Conclusion

This thesis asks how a FinOps data platform can transform FOCUS cost-and-usage records into reliable products and provide governed analysis for decision support. The project has produced an independent synthetic-data generator, a local DuckDB and Streamlit prototype, and a Databricks/GCP implementation with separate DEV and PROD processing paths. The available evidence supports different conclusions for each research question.

For reliability, the local validation confirms a reproducible source history and reconciled January 2025 analytical outputs. The validated Daily corpus contains 608 files through August 2026; the unchanged monthly billing corpus contains eighteen months through June 2026. The versioned contract, audit model and month-replacement logic specify how the cloud platform should preserve these relationships. Historical DEV and PROD processing is reported in the Belgium plan, but complete run-level artifacts are still needed to establish the cloud result independently. The full Daily-to-monthly-close cycle has not yet been validated in the thesis evidence package.

For governed self-service, the local application executes against published datamarts, which demonstrates a functioning analytical interface in that environment. It does not establish that managers or analysts can use the cloud application within verified access boundaries. Identity-based filtering, direct SQL permissions and task completion require separate tests. The reported professional problem also concerns the time taken to change an existing dashboard; that process has not been measured.

For FinOps decision support, the local datamarts expose cost perspectives, service concentration and allocation coverage. These figures come from synthetic data. The high share of `Unknown` cost-center allocation shows that a reconciled charge can still be weak evidence for accountability. Cost differences cannot be called savings without validated charge semantics and a comparable baseline. The platform's own operating cost is likewise defined methodologically but has not been quantified from a complete Databricks and GCP bill.

The project therefore establishes an auditable engineering and evaluation basis for FinOps analysis, with a locally tested subset of its intended capabilities. Further cloud, access and business validation determines how far the same design can support governed organizational use.

10.2 - Learning Outcomes and Application of Master's Coursework

The project connects several areas of the master's curriculum to one data-engineering problem. Schema design and data-quality methods appear in the versioned FOCUS contract and its Bronze-to-Silver gate. Dimensional modeling appears in the Gold fact, dimensions and datamarts. Distributed processing concepts inform the Databricks implementation, whereas DuckDB provides a separate local validation setting. The distinction between these settings is methodologically significant: code that is structured for cloud execution and an observed cloud run are different forms of evidence.

The project also places analytical measures in a business context. The negative contracted-to-effective difference in the local January dataset required checking the sign and meaning of an indicator before interpreting it as a benefit. The large `Unknown` cost-center share required distinguishing numerical reconciliation from useful allocation. These examples illustrate the application of data modeling and quality concepts to FinOps questions; they do not establish that the student personally acquired a particular skill or completed a named course.

JSON fields, tags and metadata are examples of semi-structured information used in the project. Their use does not imply that a NoSQL database was deployed. Forecasting remains a possible extension; no machine-learning result is reported. The application of coursework can be assessed against the implemented components and their validation limits, while personal learning outcomes require separate first-person evidence.

10.3 - Future Work

The immediate technical extension is an evidence-backed cloud validation of the complete Daily-to-monthly-close cycle. It requires a DEV-to-PROD promotion, replay of an already processed file, reconciliation against a monthly billing candidate, and recovery from a controlled interruption. Retained run identifiers, table outputs and costs would also permit a controlled Serverless and Classic comparison. These tests address specific gaps identified in chapters 7 and 8.

Governed use requires effective permission tests for the Databricks application, exports and direct SQL access. The current source and dashboard design do not establish Row-Level Security. Representative manager and analyst tasks on authorized data would then test whether the products answer the intended questions and whether the change workflow improves compared with the recorded baseline.

Forecasting could be investigated after a target quantity or cost measure, forecast horizon and simple reference method are defined. Chronological validation on the synthetic history would test only behavior under the generator's assumptions. Evidence from representative real history would be needed before describing operational forecasting value.

Bibliography

1. FinOps Foundation. (2026). *FinOps Framework*. <https://www.finops.org/framework/>
2. FinOps Foundation. (2026). *FinOps Open Cost and Usage Specification*. <https://www.finops.org/topic/focus/>
3. FOCUS Project. (2026). *FOCUS: FinOps Open Cost and Usage Specification*. <https://focus.finops.org/>
4. FinOps Foundation. (2024). *Adopting FOCUS, the FinOps Open Cost and Usage Specification*. <https://www.finops.org/wg/adopting-focus-the-finops-open-cost-and-usage-specification/>
5. FinOps Open Cost and Usage Specification. (2026). *FOCUS_Spec GitHub repository*. https://github.com/FinOps-Open-Cost-and-Usage-Spec/FOCUS_Spec
6. FOCUS Project. (2024). *FOCUS 1.0 - Billed Cost*. <https://focus.finops.org/docs/specification/v1-0/columns/billed-cost/>
7. FOCUS Project. (2024). *FOCUS 1.0 - Contracted Cost*. <https://focus.finops.org/docs/specification/v1-0/columns/cost-and-usage/contracted-cost/>
8. FOCUS Project. (2024). *FOCUS 1.0 - List Cost*. <https://focus.finops.org/docs/specification/v1-0/columns/cost-and-usage/list-cost/>
9. Qu, Z., Dawande, M., & Janakiraman, G. (2023). Technical Note - Cloud Cost Optimization: Model, Bounds, and Asymptotics. *Operations Research*, 72(1), 132-150. <https://doi.org/10.1287/opre.2022.0362>
10. Smendowski, M., & Nawrocki, P. (2024). Optimizing multi-time series forecasting for enhanced cloud resource utilization based on machine learning. *Knowledge-Based Systems*, 304, 112489. <https://doi.org/10.1016/j.knosys.2024.112489>
11. Nawrocki, P., & Sus, W. (2022). Anomaly detection in the context of long-term cloud resource usage planning. *Knowledge and Information Systems*, 64, 2689-2711. <https://doi.org/10.1007/s10115-022-01721-5>
12. Osypanka, P., & Nawrocki, P. (2022). Resource usage cost optimization in cloud computing using machine learning. *IEEE Transactions on Cloud Computing*, 10(3), 2079-2089. <https://doi.org/10.1109/TCC.2020.3015769>
13. Sus, W., & Nawrocki, P. (2024). Signature-based Adaptive Cloud Resource Usage Prediction Using Machine Learning and Anomaly Detection. *Journal of Grid Computing*, 22, article 46. <https://doi.org/10.1007/s10723-024-09764-4>
14. Manurung, H., & Aji, R. F. (2026). Evaluating the Implementation of the FinOps Framework for Cloud Infrastructure Cost Management: A Case Study of Technology Companies in Indonesia. *Scientific Journal of Informatics*, 12(4), 709-720. <https://doi.org/10.15294/sji.v12i4.36226>
15. Cho, C.-H. (2026). FinOps-Aware Budget-Constrained Optimization for Cloud Resource Management. *Applied Sciences*, 16(7), 3302. <https://doi.org/10.3390/app16073302>

16. Databricks. (n.d.). *Billable usage system table reference*.
<https://docs.databricks.com/gcp/en/admin/system-tables/billing> (accessed 27 September 2026).
17. Databricks. (n.d.). *Monitor job costs with system tables*.
<https://docs.databricks.com/gcp/en/admin/system-tables/jobs-cost> (accessed 27 September 2026).
18. Databricks. (n.d.). *Choose compute for your workloads*.
<https://docs.databricks.com/gcp/en/compute/choose-compute> (accessed 27 September 2026).
19. Databricks. (n.d.). *Cost optimization best practices*.
<https://docs.databricks.com/gcp/en/lakehouse-architecture/cost-optimization/best-practices> (accessed 27 September 2026).
20. FOCUS Project. (2024). *FOCUS 1.0 - Effective Cost*.
<https://focus.finops.org/docs/specification/v1-0/columns/cost-and-usage/effective-cost/> (accessed 27 September 2026).
21. FinOps Foundation. (n.d.). *FinOps Phases*. <https://www.finops.org/framework/phases/> (accessed 27 September 2026).
22. Apache Parquet. (n.d.). *Overview*. <https://parquet.apache.org/docs/overview/> (accessed 27 September 2026).
23. Databricks. (n.d.). *What is Delta Lake in Databricks?* <https://docs.databricks.com/gcp/en/delta/> (accessed 27 September 2026).
24. Microsoft. (n.d.). *Use Performance Analyzer to examine report performance*.
<https://learn.microsoft.com/en-us/power-bi/create-reports/desktop-performance-analyzer> (accessed 27 September 2026).
25. Microsoft. (n.d.). *FOCUS cost and usage details file schema*.
<https://learn.microsoft.com/en-us/azure/cost-management-billing/dataset-schema/cost-usage-details-focus> (accessed 28 September 2026).
26. Google Cloud. (n.d.). *What are data products?* <https://cloud.google.com/discover/what-are-data-products> (accessed 29 September 2026).
27. Dehghani, Z. (2020, 3 December). *Data Mesh Principles and Logical Architecture*. MartinFowler.com. Article. (accessed 15 September 2026).